

WEEK 03 MODULE
Programming Practice

TOPIC : 1 HEAP SORT

a) Easy

que. 2 **What is Sorting?**

- Sorting is the technique of arranging data elements in a particular order according to a chosen criterion, such as numerical value, alphabetical order, or date.

Types of Sorting :-

- i. Bubble Sort
- ii. Selection Sort
- iii. Insertion Sort
- iv. Merge Sort
- v. Quick Sort
- vi. Heap Sort

Heap Sort

Definition : Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap to sort elements.

Idea:

1. Build a Max Heap from the given array (largest element becomes the root).
2. Swap the root (largest element) with the last element.
3. Reduce the heap size by one and restore the Max Heap.
4. Repeat until all elements are sorted.

Key Points:-

- Uses a Binary Max Heap.
- The largest element is always at the root.
- Repeatedly places the largest element at the end of the array.
- Uses the Heapify operation to maintain the heap property.
- In-place sorting algorithm.
- Not a stable sorting algorithm.
- Time Complexity: $O(n \log n)$ (Best, Average, and Worst Case).
- Space Complexity: $O(1)$.

How It Works:-

Array: 4 10 3 5 1

Step 1: Build Max Heap	Step 2: Swap Root with Last	Step 3: Swap Root with Last	Step 4: Swap Root with Last	Step 5: Swap Root with Last
<pre> 10 / \ 5 3 /\ 4 1</pre>	<pre> 1 / \ 5 3 /\ 4 1</pre> Sorted: 10 Heapify: <pre> 5 / \ 4 3 /\ 1 1</pre>	<pre> 1 / \ 4 3</pre> Sorted: 5 10 Heapify: <pre> 4 / \ 1 3</pre>	<pre> 3 / 1</pre> Sorted: 4 5 10	<pre> 1</pre> Sorted: 3 4 5 10

Problem Statement

An online gaming platform is hosting a national-level tournament involving thousands of participants. At the end of the tournament, player scores are collected from multiple game servers and stored in random order.

Before announcing the final rankings, the organizers need all scores sorted in ascending order. Since the leaderboard data can be very large, they have decided to use **Heap Sort**, which provides guaranteed $O(N \log N)$ performance.

Your task is to implement Heap Sort and arrange all player scores in increasing order.

The sorted output will help tournament officials generate rankings and determine qualification thresholds for future events.

1. Question Analysis

An online gaming platform is conducting a national-level tournament with thousands of participants, where player scores collected from multiple game servers are stored in random order. Before announcing the final rankings, the organizers need to arrange all player scores in ascending order so that the leaderboard can be generated accurately and qualification thresholds for future events can be determined.

Since the tournament data can be very large, an efficient sorting algorithm with guaranteed performance is required. Therefore, the task is to implement the **Heap Sort** algorithm to sort the player scores in ascending order without using any built-in sorting methods, helping tournament officials generate rankings and determine qualification thresholds for future events.

2. Problem Analysis

The given problem requires sorting an unsorted list of player scores in ascending order using the Heap Sort algorithm. Since the online gaming platform manages thousands of tournament participants, the sorting technique should efficiently handle large datasets while maintaining consistent performance.

Heap Sort solves this problem by first converting the given array into a Max Heap, where the highest score is placed at the root. The root element is then repeatedly swapped with the last unsorted element, the heap size is reduced, and the remaining elements are heapified again to restore the Max Heap property. This process continues until all player scores are arranged in ascending order, enabling tournament officials to generate accurate rankings and determine qualification thresholds.

3. Algorithm Selection Analysis

Selected Algorithm: Heap Sort

- Heap Sort is based on the Binary Heap data structure.
- It first converts the given array into a Max Heap.
- The largest score is repeatedly moved to its correct sorted position.
- It guarantees a time complexity of $O(N \log N)$ in the best, average, and worst cases.
- It performs sorting in-place, requiring only constant extra memory.
- It efficiently handles large datasets containing thousands of player scores.
- Hence, Heap Sort is the most appropriate algorithm for solving this problem.

4. Points to Remember

- Heap Sort uses the Binary Heap data structure.
- A Max Heap is built before the sorting process begins.
- In a Max Heap, every parent node is greater than or equal to its child nodes.
- The largest element always remains at the root of the heap.
- The root element is repeatedly swapped with the last unsorted element.
- After every swap, the heap is rebuilt using the `heapify()` method.

- The heap size decreases after each iteration until the array becomes completely sorted.
- For any element at index i:-

Left Child Index = $2 * i + 1$

Right Child Index = $2 * i + 2$

Parent Index = $(i - 1) / 2$

- Heap Sort performs sorting in-place, requiring very little extra memory.
- Sorting is achieved by continuously swapping the root with the last unsorted element and shrinking the active heap size.
- Time Complexity
 - Best Case : $O(N \log N)$
 - Average Case : $O(N \log N)$
 - Worst Case : $O(N \log N)$
- Space Complexity : $O(1)$

5. Core Idea / Logic

- Convert the given array of player scores into a **Max Heap**.
- Place the highest score at the root of the heap.
- Swap the root element with the last unsorted element.
- Reduce the heap size by one.
- Apply the heapify() method to restore the Max Heap property.
- Repeat the swapping and heapifying process until only one element remains in the heap.
- Display the final sorted player scores in ascending order.

6. Execution Flow

Part A : Execution Flow

Input Phase

1. The program starts execution from the main() method.
2. The user enters the number of player scores (N), each of which is validated before being stored in the array.
3. The program validates whether N is within the specified range.
4. An integer array of size N is created.
5. The user enters all player scores.
6. The entered scores are stored in the array.
7. The heapSort() method is called to sort the array.

Processing Phase

1. The heapSort() method converts the given array into a **Max Heap**.
2. The largest element is placed at the root of the heap.
3. The root element is swapped with the last unsorted element.
4. The heap size is reduced by one.
5. The heapify() method is called to restore the Max Heap property.
6. The swapping and heapifying process is repeated until all elements are sorted.
7. The array becomes completely sorted in ascending order.

Output Phase

1. Control returns to the main() method.
2. The sorted array is traversed.
3. The player scores are displayed in ascending order.
4. The program terminates successfully.

Part B : Detailed Steps

Step 1: Start the execution of the program.

Step 2: Read the number of player scores (N) from the user.

Step 3: Validate the value of N. If N is less than 1 or greater than 100000, display an error message and terminate the program.

Step 4: Create an integer array of size N to store the player scores.

Step 5: Read and store all the player scores in the array one by one, validate each score according to the specified constraints, and store the valid scores in the array.

Step 6: Call the heapSort(arr) method to sort the array.

Step 7: Build a Max Heap from the given array by calling the heapify() method for all non-leaf nodes.

Step 8: Swap the root element (largest element) with the last unsorted element of the heap.

Step 9: Reduce the heap size by one so that the last element is excluded from further heap operations.

Step 10: Call the heapify() method to restore the Max Heap property for the reduced heap.

Step 11: Repeat the swapping and heapifying process until the heap contains only one element.

Step 12: Display the sorted player scores in ascending order.

Step 13: Close the Scanner object and terminate the program.

7. Program Structure Analysis

i. Variable n

int n;

Purpose:

- Stores the total number of player scores entered by the user.
- Determines the size of the array.
- Defines the number of elements to be sorted.

Constraint:

- $1 \leq n \leq 100000$

In simple words:

- Minimum value of n can be 1.

Maximum value of n can be 100000.

ii. Array arr[]

```
int[] arr = new int[n];
```

Purpose:

- Stores all player scores.
- Acts as the main array on which Heap Sort is performed.
- Contains the sorted player scores after the sorting process is completed.

Constraint:

- $0 \leq \text{Score} \leq 1000000$

In simple words:

- Minimum player score can be 0.
- Maximum player score can be 1000000.

iii. Variable largest

```
int largest = i;
```

Purpose:

- Stores the index of the largest element in the current subtree.
- Initially assumes the root node is the largest.
- Gets updated if either child node contains a larger value.

iv. Variables left and right

```
int left = 2 * i + 1;
int right = 2 * i + 2;
```

Purpose:

- left stores the index of the left child of the current node.
- right stores the index of the right child of the current node.
- Used to compare the child nodes with the parent node during heapification.

v. Variable temp

```
int temp;
```

Purpose:

- Temporarily stores an element during the swapping process.
- Helps exchange the positions of two elements without losing any data.

vi. Input Loop

```
for(int i = 0; i < n; i++)  
{  
    arr[i] = sc.nextInt();  
}
```

Purpose:

- Accepts all player scores from the user.
- Stores them in the array.
- Ensures all input values are available before sorting begins.

vii. heapSort() Method

```
heapSort(arr);
```

Purpose:

- Performs Heap Sort on the given array.
- Builds a Max Heap from the input array.
- Repeatedly places the largest element at its correct sorted position.
- Produces the final sorted array in ascending order.

viii. heapify() Method

```
heapify(arr, n, i);
```

Purpose:

- Restores the Max Heap property for a subtree.
- Compares the parent node with its left and right child nodes.
- Swaps elements when required.
- Recursively heapifies the affected subtree.

ix. Max Heap Construction Loop

```
for(int i = n / 2 - 1; i >= 0; i--)
```

Purpose:

- Starts from the last non-leaf node.
- Calls the heapify() method for each non-leaf node.
- Converts the entire array into a Max Heap.

x. Extraction Loop

```
for(int i = n - 1; i > 0; i--)
```

Purpose:

- Repeatedly swaps the root element with the last unsorted element.
- Reduces the heap size after each swap.
- Calls the heapify() method to rebuild the Max Heap.
- Continues until the entire array becomes sorted.

xi. Left Child Comparison

```
if(left < n && arr[left] > arr[largest])
```

Purpose:

- Checks whether the left child exists.
- Compares the left child with the current largest element.
- Updates the largest index if the left child has a greater value.

xii. Right Child Comparison

```
if(right < n && arr[right] > arr[largest])
```

Purpose:

- Checks whether the right child exists.
- Compares the right child with the current largest element.
- Updates the largest index if the right child has a greater value.

xiii. Swap Condition

```
if(largest != i)
```

Purpose:

- Checks whether a child node contains a larger value than the parent node.
- Swaps the parent with the largest child.
- Calls heapify() recursively to maintain the Max Heap property.

xiv. Output Loop

```
for(int i = 0; i < n; i++)  
{  
    System.out.print(arr[i] + " ");  
}
```

Purpose:

- Traverses the sorted array.
- Displays all player scores in ascending order.
- Produces the final sorted output.

8. Program

```
package week3_module;  
import java.util.Scanner;  
public class HeapSortEasy {  
  
    // Function to heapify a subtree rooted at index i  
    static void heapify(int[] arr, int n, int i) {  
        int largest = i;  
        int left = 2 * i + 1;  
        int right = 2 * i + 2;  
  
        // Check left child  
  
        if (left < n && arr[left] > arr[largest]) {  
            largest = left;  
        }  
    }  
}
```

```

    }

    // Check right child
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // If largest is not the root
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        // Heapify the affected subtree
        heapify(arr, n, largest);
    }
}

// Heap Sort function
static void heapSort(int[] arr) {
    int n = arr.length;

    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    // Extract elements one by one
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // Heapify the reduced heap
        heapify(arr, i, 0);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter the number of player scores to be sorted : ");
    int n = sc.nextInt();
    if (n < 1 || n > 100000) {
        System.out.println("Error: n must be between 1 and 100000.");
        sc.close();
        return;
    }
    int[] arr = new int[n];
    System.out.print("Enter the " + n + " player scores collected randomly from multiple game servers: ");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
        if (arr[i] < 0 || arr[i] > 1000000) {
            System.out.println("Error: Score must be between 0 and 1000000.");
            sc.close();
            return;
        }
    }
    heapSort(arr);

```

```

    System.out.print("Sorted player scores : ");

```

```

        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + " ");
        }
        sc.close();
    }
}

```

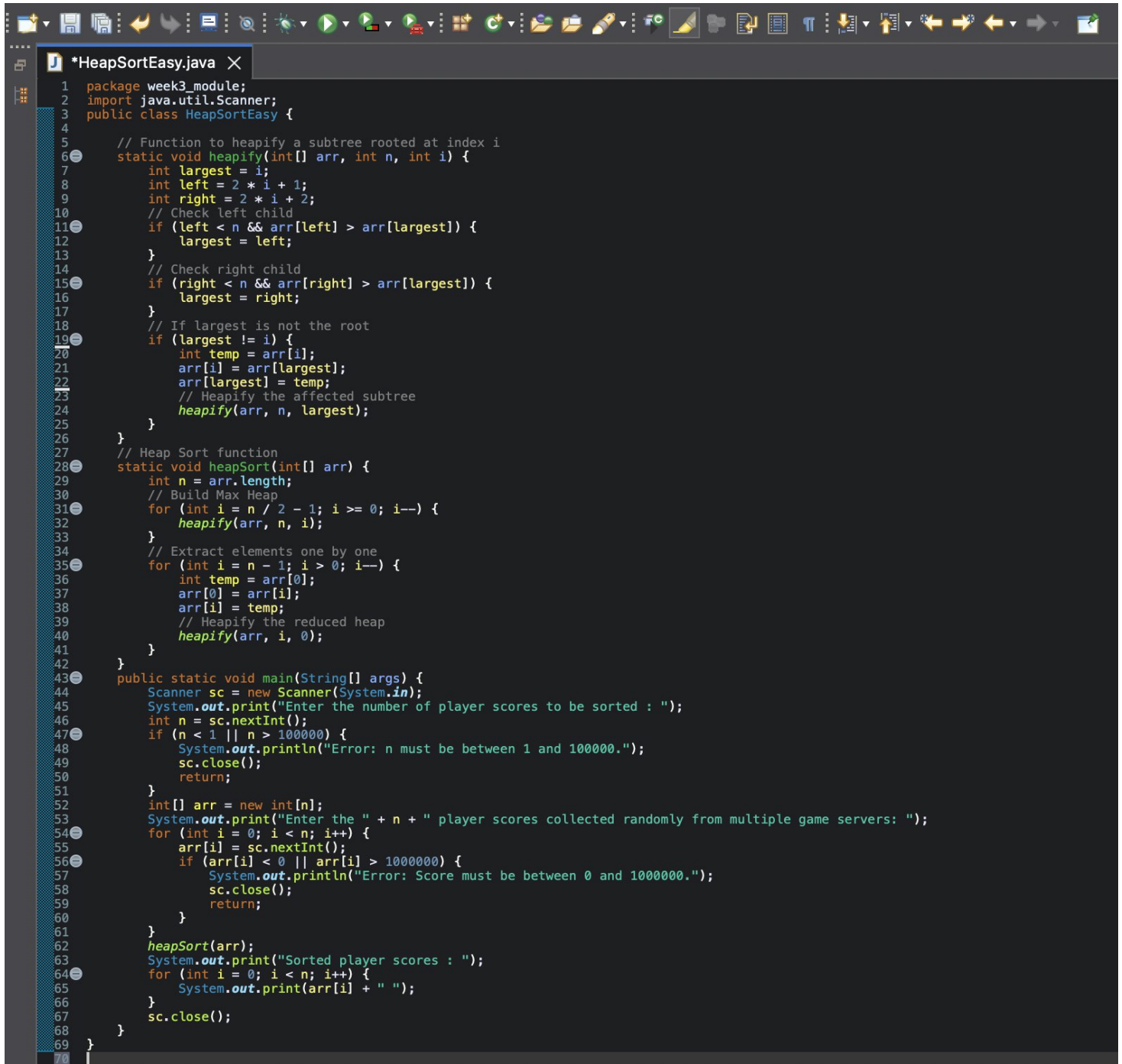
9. Program's output

Enter the number of player scores to be sorted : 6

Enter the 6 player scores collected randomly from multiple game servers: 850 1200 950 600 1500 1000

Sorted player scores : 600 850 950 1000 1200 1500

10. Screenshot program

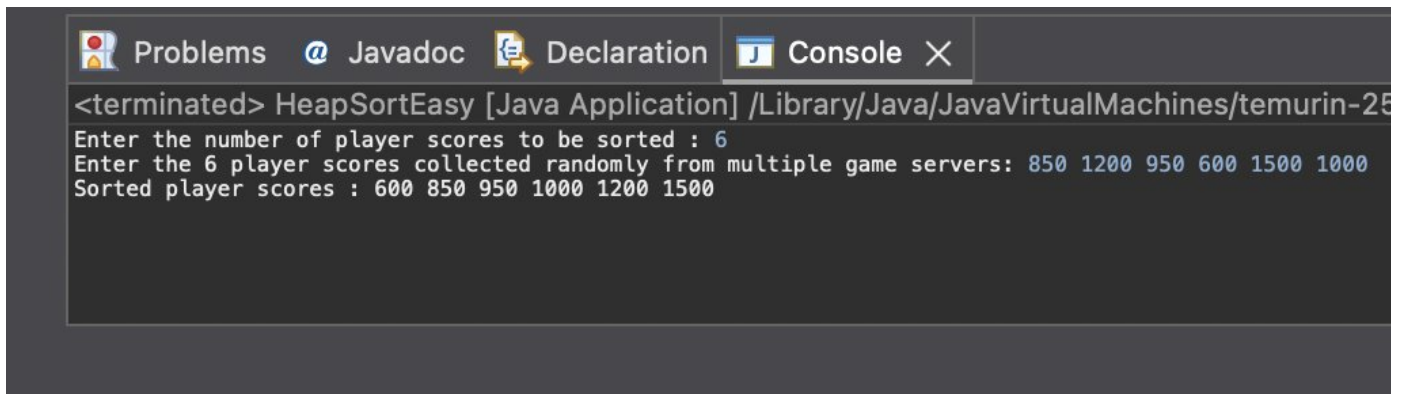


```

1 package week3_module;
2 import java.util.Scanner;
3 public class HeapSortEasy {
4
5     // Function to heapify a subtree rooted at index i
6     static void heapify(int[] arr, int n, int i) {
7         int largest = i;
8         int left = 2 * i + 1;
9         int right = 2 * i + 2;
10        // Check left child
11        if (left < n && arr[left] > arr[largest]) {
12            largest = left;
13        }
14        // Check right child
15        if (right < n && arr[right] > arr[largest]) {
16            largest = right;
17        }
18        // If largest is not the root
19        if (largest != i) {
20            int temp = arr[i];
21            arr[i] = arr[largest];
22            arr[largest] = temp;
23            // Heapify the affected subtree
24            heapify(arr, n, largest);
25        }
26    }
27    // Heap Sort function
28    static void heapSort(int[] arr) {
29        int n = arr.length;
30        // Build Max Heap
31        for (int i = n / 2 - 1; i >= 0; i--) {
32            heapify(arr, n, i);
33        }
34        // Extract elements one by one
35        for (int i = n - 1; i > 0; i--) {
36            int temp = arr[0];
37            arr[0] = arr[i];
38            arr[i] = temp;
39            // Heapify the reduced heap
40            heapify(arr, i, 0);
41        }
42    }
43    public static void main(String[] args) {
44        Scanner sc = new Scanner(System.in);
45        System.out.print("Enter the number of player scores to be sorted : ");
46        int n = sc.nextInt();
47        if (n < 1 || n > 100000) {
48            System.out.println("Error: n must be between 1 and 100000.");
49            sc.close();
50            return;
51        }
52        int[] arr = new int[n];
53        System.out.print("Enter the " + n + " player scores collected randomly from multiple game servers: ");
54        for (int i = 0; i < n; i++) {
55            arr[i] = sc.nextInt();
56            if (arr[i] < 0 || arr[i] > 1000000) {
57                System.out.println("Error: Score must be between 0 and 1000000.");
58                sc.close();
59                return;
60            }
61        }
62        heapSort(arr);
63        System.out.print("Sorted player scores : ");
64        for (int i = 0; i < n; i++) {
65            System.out.print(arr[i] + " ");
66        }
67        sc.close();
68    }
69 }
70

```

program's output



```
<terminated> HeapSortEasy [Java Application] /Library/Java/JavaVirtualMachines/temurin-25
Enter the number of player scores to be sorted : 6
Enter the 6 player scores collected randomly from multiple game servers: 850 1200 950 600 1500 1000
Sorted player scores : 600 850 950 1000 1200 1500
```

11. EXPLANATION :-

Program Breakdown

Execution always starts from the **main()** method.

```
public static void main(String[] args)
```

Control first enters the main() method.

Step 1 : Create Scanner Object

```
Scanner sc = new Scanner(System.in);
```

Purpose

Creates a Scanner object to accept input from the keyboard.

Step 2 : Read Number of Player Scores

```
int n = sc.nextInt();
```

Suppose the user enters

6

Then

$n = 6$

This means **6 player scores** will be entered and sorted.

Step 3 : Validate N

```
if(n < 1 || n > 100000)
```

$1 \leq N \leq 100000$

If the entered value is outside this range,

```
System.out.println("Error...");
sc.close();
return;
```

the program displays an error message and terminates.

Step 4 : Create Array

```
int[] arr = new int[n];
if(arr[i] < 0 || arr[i] > 1000000)
```

Purpose:

- Checks whether the entered player score satisfies the given constraints.
- Displays an error message if an invalid score is entered.
- Terminates the program to prevent invalid data from being processed.

Step 5 : Read Player Scores

```
for(int i=0;i<n;i++)
{
    arr[i]=sc.nextInt();
}
```

Suppose input is

850 1200 950 600 1500 1000

Array becomes

850 1200 950 600 1500 1000

Step 6 : Call Heap Sort

```
heapSort(arr);
```

Control now moves from main() to heapSort()

heapSort() Method

```
static void heapSort(int[] arr)
```

This method performs the complete Heap Sort.

Step 7 : Find Array Size

```
int n = arr.length;
```

If arr = [850 1200 950 600 1500 1000]

then

n = 6

Step 8 : Build Max Heap

```
for(int i=n/2-1;i>=0;i--)
```

This loop builds a **Max Heap**.

Max Heap:->

A Max Heap is a Binary Tree in which

Parent $\geq$ Left Child

Parent $\geq$ Right Child

Example

```
      1500
     /  \
    1200 1000
   / \  /
  600 850 950
```

Notice

$1500 > 1200$

$1500 > 1000$

$1200 > 600$

$1200 > 850$

$1000 > 950$

Every parent is greater than its children.

Why

$n/2-1$

Suppose $n=6$ Then $6/2-1$

$3-1 = 2$

Loop becomes

$i=2$

$i=1$

$i=0$

Why only these?

Because

0 , 1 , 2

are the **non-leaf nodes**.

Leaf nodes already satisfy the heap property because they have no children.

Therefore,

Heap construction always starts from the **last non-leaf node** and moves upward to the root.

Step 9 : Call heapify()

Inside the loop

```
heapify(arr,n,i);
```

is called.

heapify() Method

```
static void heapify(int[] arr,int n,int i)
```

Purpose

Restore the Max Heap property for the subtree rooted at index i.

Step 10 : Assume Root is Largest

```
int largest=i;
```

Initially,

the program assumes

Parent is the largest element.

Example

950

Parent index

```
i=2
```

Then, largest=2

Step 11 : Calculate Left Child

```
int left=2*i+1;
```

Suppose i=2 Then

```
left=5
```

Formula : Left Child = $2*i+1$

is always used in arrays representing binary heaps.

Step 12 : Calculate Right Child

`int right=2*i+2;`

Suppose $i=2$ Then

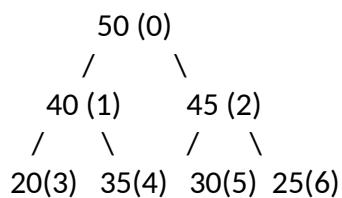
`right=6`

Formula : Right Child = $2*i+2$

Suppose the array is

Index	0	1	2	3	4	5	6
Value	50	40	45	20	35	30	25

Representation



Notice

For index **0**

`Left = 2×0+1=1`

`Right = 2×0+2=2`

For index **1**

`Left = 2×1+1=3`

`Right = 2×1+2=4`

For index **2**

`Left = 2×2+1=5`

`Right = 2×2+2=6`

Step 13 : Compare Left Child

`if(left<n && arr[left]>arr[largest])`

Meaning

Does left child exist? AND Is left child larger?

If yes,

`largest=left;`

Now left child becomes the largest.

Step 14 : Compare Right Child

```
if(right<n && arr[right]>arr[largest])
```

Same process.

Does right child exist? AND Is right child larger?

If right child is even larger,

largest=right;

Step 15 : Swap Parent and Largest Child

```
if(largest!=i)
```

Meaning

Parent is **not** the largest.

Swap them.

```
int temp=arr[i];
```

```
arr[i]=arr[largest];
```

```
arr[largest]=temp;
```

Now the larger value comes to the parent.

Step 16 : Recursive heapify()

```
heapify(arr,n,largest);
```

we are doing the same again because after swapping, the child subtree may no longer satisfy the Max Heap property. So the program recursively heapifies that subtree until it also becomes a valid Max Heap.

After Build Heap

Eventually, array becomes a Max Heap.

Example : 1500 1200 1000 600 850 950

Largest element is now = 1500 at index = 0

Step 17 : Extraction Loop

```
for(int i=n-1;i>0;i--)
```

Purpose

Move the largest element to its correct sorted position.

Step 18 : Swap Root with Last Element

`arr[0] ↔ arr[i]`

Example

Before :- 1500 1200 1000 600 850 950

Swap

1500

↓

last position

After :- 950 1200 1000 600 850 1500

Notice 1500 is now permanently sorted.

Step 19 : Reduce Heap Size

`heapify(arr,i,0);`

Notice i is passed instead of n because the last element 1500 is already sorted. We don't want `heapify()` to touch it again. So the heap size becomes 5 instead of 6

Step 20 : Repeat

Again Largest comes to top.

Swap.

Reduce heap.

Heapify.

Repeat.

Eventually

600 850 950 1000 1200 1500

Step 21 : Return to main()

When Heap Sort finishes, control returns to `main()`

Step 22 : Print Sorted Array

`for(int i=0;i<n;i++)`

Output : 600 850 950 1000 1200 1500

Step 23 : Close Scanner

`sc.close();`

Scanner is closed.

Program terminates successfully.

Overall Program Flow

main()



Read N



Validate N



Create Array



Read Player Scores



Call heapSort()



Build Max Heap



Call heapify()



Swap Root with Last Element



Reduce Heap Size



Heapify Remaining Heap



Repeat Until Heap Size Becomes 1



Return to main()



Display Sorted Scores



Close Scanner and terminate the program